

Back-End Development Fundamentals

Chapters 6.1 · 6.2 · 6.3 | 40 Multiple Choice Questions

Section 1: Full-Stack Development & Node.js Basics

Q1. What is Node.js?

- A) A front-end framework for building user interfaces
- B) A JavaScript runtime environment that runs JavaScript outside the browser
- C) A CSS preprocessor for styling web pages
- D) A database management system

Answer: B

Q2. Which of the following best describes the role of Express.js in back-end development?

- A) It is a database driver for connecting to SQL databases
- B) It is a JavaScript runtime that replaces Node.js
- C) It is a minimal web framework built on Node.js that simplifies creating routes and APIs
- D) It is a front-end library for rendering components

Answer: C

Q3. Before Node.js existed, which languages were commonly used for back-end development? (Choose the BEST answer)

- A) HTML, CSS, and JavaScript
- B) Python, Java, PHP, and other non-JavaScript languages
- C) React, Angular, and Vue
- D) TypeScript and Dart only

Answer: B

Q4. Which HTTP method is used to REQUEST data from the server?

- A) POST
- B) DELETE
- C) PUT
- D) GET

Answer: D

Q5. Which HTTP method is used to SEND data to the server?

- A) GET
- B) POST
- C) FETCH
- D) READ

Answer: B

Q6. In a full-stack application, which part handles business logic, databases, and APIs?

- A) The front-end (client-side)
- B) The browser's DOM
- C) The back-end (server-side)
- D) React components

Answer: C

Section 2: Express Routes & Request/Response

Q7. Which of the following correctly defines a GET route in Express that returns a JSON array of students?

- A)

```
app.get('/api/students', (req, res) => {  
  res.json(students);  
});
```
- B)

```
app.post('/api/students', (req, res) => {  
  res.json(students);  
});
```
- C)

```
app.get('/api/students', (req, res) => {  
  req.json(students);  
});
```
- D)

```
app.fetch('/api/students', (req, res) => {  
  res.send(students);  
});
```

Answer: A

Q8. Which Express code snippet correctly handles a POST request that adds a new student from the request body?

- A)

```
app.get('/api/students', (req, res) => {  
  students.push(req.body);  
  res.json(req.body);  
});
```
- B)

```
app.post('/api/students', (req, res) => {  
  const newStudent = req.body;  
  students.push(newStudent);  
  res.json(newStudent);  
});
```
- C)

```
app.post('/api/students', (req, res) => {  
  const newStudent = res.body;  
  students.push(newStudent);  
  res.json(newStudent);  
});
```
- D)

```
app.put('/api/students', (req, res) => {  
  students.push(req.params);  
  res.json(students);  
});
```

Answer: B

Q9. In Express, what does the 'req' parameter represent?

- A) The response that the server sends back
- B) The request from the front-end (browser)
- C) A required import from the Express library
- D) A database query object

Answer: B

Q10. Which front-end fetch call correctly sends a GET request to retrieve all students from the back-end?

- A) `fetch('http://localhost:3000/api/students', {method: 'POST'})`
`.then(r => r.json())`
`.then(data => setStudents(data));`
- B) `fetch('http://localhost:3000/api/students')`
`.then(r => r.json())`
`.then(data => setStudents(data));`
- C) `fetch('http://localhost:3000/api/students')`
`.then(r => r.send())`
`.then(data => setStudents(data));`
- D) `get('http://localhost:3000/api/students')`
`.then(r => r.json())`
`.then(data => setStudents(data));`

Answer: B

Q11. What is a 'route' in the context of Express.js?

- A) A CSS path for styling components
- B) A server path that handles incoming requests at a specific URL
- C) A navigation component in React
- D) A method for connecting to a database

Answer: B

Q12. What is the correct way to start a Node.js server file named 'server.js'?

- A) `npm server.js`
- B) `run server.js`
- C) `node server.js`
- D) `start server.js`

Answer: C

Section 3: npm, package.json & Project Setup

Q13. What is npm?

- A) Node's built-in module system
- B) A JavaScript runtime environment
- C) The default package manager for Node.js used to install and manage packages
- D) A front-end framework for building SPAs

Answer: C

Q14. Which command initializes a new Node.js project and creates a package.json with default values without interactive prompts?

- A) npm init
- B) npm start
- C) npm init -y
- D) node init --default

Answer: C

Q15. Which command correctly installs a package called 'lodash' in a Node.js project?

- A) node install lodash
- B) npm install lodash
- C) npm get lodash
- D) require install lodash

Answer: B

Q16. What does package.json NOT typically contain?

- A) Project name and version
- B) Project dependencies
- C) Project scripts
- D) The actual source code of installed packages

Answer: D

Q17. What is the purpose of package-lock.json?

- A) To lock the project so other developers cannot edit it
- B) To record the exact versions of all installed packages for consistent installs across machines
- C) To store the user's npm account credentials
- D) To list available npm commands

Answer: B

Q18. A developer runs: npm install --save-dev jest What does this do?

- A) Installs jest globally on the machine
- B) Installs jest as a regular dependency listed under 'dependencies'
- C) Installs jest as a development dependency listed under 'devDependencies'
- D) Uninstalls jest from the project

Answer: C

Q19. Which command verifies that Node.js is installed correctly on your machine?

- A) node --check
- B) node -v
- C) npm verify
- D) node status

Answer: B

Section 4: Command Line Arguments

Q20. Given the command: `node script.js add 5 10` What is the value of `process.argv[2]`?

- A) The path to Node.js
- B) The path to script.js
- C) "add"
- D) "5"

Answer: C

Q21. What is `process.argv[0]` always set to when running a Node.js script?

- A) The first argument passed by the user
- B) The name of the script file
- C) The path to the Node.js executable
- D) undefined

Answer: C

Q22. A student writes this code: `const num = process.argv[3]; console.log(num + 10);` Then runs: `node calc.js add 5` What will be printed?

- A) 15
- B) 510
- C) NaN
- D) An error is thrown

Answer: B

Q23. Which code correctly converts a command-line argument to a number for arithmetic?

- A) `const num = process.argv[2];`
- B) `const num = parseInt(process.argv[2]);`
- C) `const num = process.argv.toString();`
- D) `const num = Number.argv[2];`

Answer: B

Section 5: ES Modules (import / export)

Q24. Which of the following correctly exports a function named 'add' from a module?

- A)

```
module.add = function(numbers) {  
  return numbers.reduce((s, n) => s + n, 0);  
}
```
- B)

```
export function add(numbers) {  
  return numbers.reduce((s, n) => s + n, 0);  
}
```
- C)

```
exports.add = function(numbers) {  
  return numbers.reduce((s, n) => s + n, 0);  
}
```
- D)

```
send function add(numbers) {  
  return numbers.reduce((s, n) => s + n, 0);  
}
```

Answer: B

Q25. Which import statement correctly imports the 'add' and 'subtract' functions from './utils/operations.js'?

- A) `import add, subtract from './utils/operations.js';`
- B) `import { add, subtract } from './utils/operations.js';`
- C) `import { add, subtract } from './utils/operations';`
- D) `require({ add, subtract }, './utils/operations.js');`

Answer: B

Q26. Why must you include the .js extension when importing a local module in ES Modules?

- A) It's optional — Node.js will find the file without it
- B) ES Modules require explicit file extensions so Node.js can locate the correct file
- C) It prevents the file from being cached
- D) It enables TypeScript support

Answer: B

Q27. Which import correctly imports a default export (like lodash) from an installed npm package?

- A) `import { _ } from 'lodash';`
- B) `import lodash from 'lodash';`
- C) `import lodash from './lodash.js';`
- D) `require _ from 'lodash';`

Answer: B

Q28. What are the TWO module formats supported in Node.js?

- A) ESM (import/export) and CommonJS (require/module.exports)
- B) AMD and UMD
- C) TypeScript modules and Babel modules
- D) Named exports and default exports only

Answer: A

Q29. A project has this structure: calculator.js utils/ operations.js From calculator.js, which path correctly imports from operations.js?

- A) `import { add } from 'operations.js';`
- B) `import { add } from '../operations.js';`
- C) `import { add } from './utils/operations.js';`
- D) `import { add } from '/utils/operations.js';`

Answer: C

Q30. What is the main benefit of organizing code into modules?

- A) It makes the code run faster by compiling to machine code
- B) It allows code organization, reusability, and separation of concerns — each file has one clear purpose
- C) It automatically handles all HTTP requests
- D) It removes the need for npm packages

Answer: B

Section 6: Integration & Advanced Concepts

Q31. A developer creates a module `parser.js` that imports `lodash` and exports a `parseNumbers` function. Which code in `calculator.js` correctly uses it?

- A) `import parseNumbers from './utils/parser.js';`
`const nums = parseNumbers(['5', '10', '15']);`
- B) `import { parseNumbers } from './utils/parser.js';`
`const nums = parseNumbers(['5', '10', '15']);`
- C) `require { parseNumbers } from './utils/parser.js';`
`const nums = parseNumbers(['5', '10', '15']);`
- D) `import { parseNumbers } from 'parser.js';`
`const nums = parseNumbers(['5', '10', '15']);`

Answer: B

Q32. Which of the following best describes the flow when a React front-end fetches data from an Express back-end?

- A) Back-end sends a request → Front-end processes it → Back-end receives the response
- B) Front-end sends a GET request → Express processes it → Express sends JSON response → Front-end updates the UI
- C) The database directly pushes data to React without a server
- D) Express renders HTML and sends it to the browser

Answer: B

Q33. What does `'npm list lodash'` do?

- A) Installs the `lodash` package
- B) Removes `lodash` from the project
- C) Checks whether `lodash` is installed in the current project
- D) Opens the `lodash` documentation

Answer: C

Q34. Examine the following code: `import _ from 'lodash'; const numbers = [1, 2, 3, 4, 5]; const result = _.sum(numbers);` What is the value of `result`?

- A) `[1, 2, 3, 4, 5]`
- B) `'12345'`
- C) `15`
- D) `NaN`

Answer: C

Q35. A student runs: `node test.js hello` What does `console.log(process.argv[1])` print?

- A) `'hello'`
- B) The path to the Node.js executable
- C) The full path to `test.js`
- D) `undefined`

Answer: C

Q36. Which of the following is the INCORRECT way to import a local module?

- A) `import { add } from './utils/operations.js';`
- B) `import { add } from '../utils/operations.js';`
- C) `import { add } from './utils/operations';`
- D) `import _ from 'lodash';`

Answer: C

Q37. A team is collaborating on a Node.js project. Developer A installs `lodash@4.17.21` and pushes `package-lock.json`. What happens when Developer B runs `'npm install'`?

- A) npm installs the latest version of `lodash` regardless of `package-lock.json`
- B) npm installs the exact same version (4.17.21) as specified in `package-lock.json`
- C) npm throws an error because `lodash` is already installed
- D) npm deletes the existing packages and starts fresh

Answer: B

Q38. Consider this Express server. A browser visits `http://localhost:3000/api/students`. Which option correctly describes what happens? `app.get('/api/students', (req, res) => { res.json([ {id:1, name:'Ali'}, {id:2, name:'Sara'}]); });`

- A) The server crashes because no database is connected
- B) The browser receives a JSON array with two student objects
- C) The server sends an HTML page to the browser
- D) Nothing happens because GET requests cannot return JSON

Answer: B

Q39. Which module design approach follows the 'single responsibility' best practice?

- A) Putting all math operations, input parsing, and validation into one file called `utils.js`
- B) Creating `operations.js` for math only, `parser.js` for input parsing, and `validator.js` for validation
- C) Putting all code into `server.js` to reduce the number of files
- D) Using one large `index.js` that imports nothing

Answer: B

Q40. A developer builds a CLI calculator with this command: `node calculator.js multiply 6 7` Which code correctly extracts and multiplies the two numbers?

- A) `const a = process.argv[2];`
`const b = process.argv[3];`
`console.log(a * b);`
- B) `const a = Number(process.argv[3]);`
`const b = Number(process.argv[4]);`
`console.log(a * b);`
- C) `const a = process.argv[0];`
`const b = process.argv[1];`
`console.log(a * b);`
- D) `const a = parseInt(process.argv[1]);`
`const b = parseInt(process.argv[2]);`
`console.log(a * b);`

Answer: B

SWE 363 | KFUPM | Practice Questions

SWE 363 — Web Engineering & Development

Back-End Development: Node.js & Express.js

Multiple Choice Questions (Lectures 17 & 18)

Choose the best answer for each question. • 40 Questions Total

Section 1: Express.js Fundamentals & Setup

1. What is Express.js?

- A) A database management system for Node.js
- B) A web framework for Node.js that simplifies building web servers
- C) A front-end JavaScript library similar to React
- D) A built-in Node.js module for handling HTTP requests

2. Which command correctly installs Express and CORS for a new project?

- A) `npm install express cors`
- B) `node install express cors`
- C) `npm add express && npm add cors`
- D) `npm init express cors`

3. Which of the following correctly creates and starts an Express server on port 3000?

- A) `const app = express(); app.start(3000);`
- B) `const app = express(); app.listen(3000, () => console.log('running'));`
- C) `const app = new Express(); app.run(3000);`
- D) `const app = express(); app.connect(3000);`

4. What does `npm init -y` do when setting up a Node.js project?

- A) Installs all dependencies listed in package.json
- B) Initializes a new Git repository
- C) Creates a package.json with default values without prompting
- D) Starts the Node.js server with default settings

5. What is the role of `package-lock.json`?

- A) It stores environment variables for the project
- B) It lists the scripts you can run with npm
- C) It records exact versions of all installed packages for consistent installs
- D) It defines the main entry point of the application

Section 2: Routes & HTTP Methods

6. What is the correct syntax for defining a route in Express?

- A) `app.route(METHOD, path, handler)`
- B) `app.METHOD(path, handler)`
- C) `app.handle(path, METHOD, handler)`
- D) `route.app(METHOD, path, handler)`

7. Which HTTP method should be used to retrieve data from an API endpoint?

- A) POST
- B) PUT
- C) GET
- D) DELETE

8. What will the following route respond with when a user visits `/api/quote`?

```
app.get('/api/quote', (req, res) => { res.json({ quote: 'Show me the code.' }); });
```

- A) A plain text string: Show me the code.
- B) A JSON object: `{ quote: 'Show me the code.' }`
- C) An HTML page displaying the quote
- D) A 404 status code because the route is missing a parameter

9. Which of the following correctly sends a 404 status with a message?

- A) `res.send(404, 'Not Found');`
- B) `res.json({ status: 404 });`
- C) `res.status(404).send('Not Found');`
- D) `res.error(404).send('Not Found');`

10. An Express server has three routes: `/`, `/api/quote`, and `/api/about`. A request is made to `/api/users`. What happens?

- A) Express matches the closest route, which is `/api/about`
- B) Express returns a default HTML page
- C) No route matches, and Express sends no response unless there is an error handler
- D) Express restarts the server

11. Which HTTP method is most appropriate for creating a new user in a REST API?

- A) GET
- B) DELETE
- C) PUT
- D) POST

12. What does `res.sendStatus(200)` do differently from `res.status(200).send('OK')`?

- A) They are completely identical in all cases
- B) `res.sendStatus(200)` sets the status and sends the status message string as the body automatically
- C) `res.sendStatus(200)` only sets the header without sending a body
- D) `res.sendStatus(200)` is not a valid Express method

Section 3: Middleware

13. What is middleware in Express.js?

- A) Functions that run only after the response is sent to the client
- B) Functions that run between a request and a response in the request-response cycle
- C) A special type of route that handles only POST requests
- D) A built-in database connector for Express

14. What is the correct way to apply global middleware in Express?

- A) `app.route(middlewareFunc);`
- B) `app.middleware(middlewareFunc);`
- C) `app.use(middlewareFunc);`
- D) `app.handle(middlewareFunc);`

15. What happens if a middleware function does NOT call `next()`?

- A) Express skips the middleware and moves to the next one automatically
- B) The request-response cycle is left hanging — no further middleware or route runs
- C) Express throws a runtime error and crashes
- D) The server sends a 500 error response automatically

16. Which of the following correctly adds a request logger middleware?

- A) `app.log((req, res) => { console.log(req.url); });`
- B) `app.use((req, res, next) => { console.log(req.url); next(); });`
- C) `app.use((req, res) => { console.log(req.url); });`
- D) `middleware((req, res, next) => { console.log(req.url); next(); });`

17. Why is CORS middleware needed when a React frontend (port 5173) talks to an Express backend (port 3000)?

- A) Because Express blocks all requests by default for security
- B) Because browsers block cross-origin requests by default, and CORS tells the browser to allow them
- C) Because Node.js cannot process JSON without CORS enabled
- D) Because the React app uses a different HTTP version than the backend

18. Which code snippet correctly enables CORS in an Express application?

- A) `import cors from 'cors'; app.enable(cors());`
- B) `import cors from 'cors'; app.use(cors());`
- C) `app.use(express.cors());`
- D) `app.cors(true);`

19. What is the correct order of middleware in the following setup? `app.use(cors());`
`app.use(express.json()); app.get('/data', handler);`

- A) Route runs first, then `express.json()`, then `cors()`
- B) `cors()` → `express.json()` → route handler (top to bottom)
- C) `express.json()` → `cors()` → route handler
- D) The order does not matter in Express

20. Which built-in Express middleware allows your server to read JSON data sent in a request body?

- A) `app.use(express.text());`
- B) `app.use(express.raw());`
- C) `app.use(express.json());`
- D) `app.use(express.body());`

Section 4: Query Parameters & Route Parameters

21. Given the URL `/search?q=nodejs&page;=3`, which code correctly reads the search term?

- A) `const q = req.params.q;`
- B) `const q = req.body.q;`
- C) `const q = req.query.q;`
- D) `const q = req.url.q;`

22. Query string values in Express are always received as which data type?

- A) Number
- B) Boolean
- C) Object
- D) String

23. How do you access the `:userId` parameter from the route `/users/:userId`?

- A) `req.query.userId`
- B) `req.params.userId`
- C) `req.body.userId`
- D) `req.url.userId`

24. What will `req.params` contain for the route `/profile/:first/:last` when the URL is `/profile/Jack/Black`?

- A) { first: 'Jack', last: 'Black' }
- B) { 0: 'Jack', 1: 'Black' }
- C) { name: 'Jack Black' }
- D) { first: 'Jack Black' }

25. Which statement best describes when to use route parameters vs query parameters?

- A) Route params are for optional filters; query params are for resource identification
- B) Route params identify a specific resource in the path; query params are for optional filters/sorting
- C) They are interchangeable — use whichever looks cleaner
- D) Query params are only for GET requests; route params work with all methods

26. A developer writes `const page = req.query.page;` and then uses it in a calculation. What bug might occur?

- A) `req.query.page` returns undefined because `page` is not a valid query key
- B) Arithmetic will fail or produce NaN because query values are strings, not numbers
- C) Express throws an error if query params are used in math
- D) The value will always be 0 because query params default to zero

27. What is the purpose of `app.param()` in Express?

- A) To define a new query parameter for all routes
- B) To run validation/conversion middleware once whenever a named route parameter is matched
- C) To set default values for missing route parameters
- D) To parse the URL and extract all parameters at once

28. Given the following code, what is stored in `req.userIdNum` after a request to `/users/42`?

```
app.param('userId', (req, res, next, userId) => { req.userIdNum =  
Number(userId); next(); });
```

- A) The string '42'
- B) The number 42
- C) undefined — `app.param()` doesn't store values on `req`
- D) An object: { `userId`: 42 }

29. If `app.param('userId', validator)` is defined, for which routes does the validator automatically run?

- A) Only for GET routes with `:userId`
- B) Only for the first route that uses `:userId`
- C) For ALL routes (GET, POST, DELETE, etc.) that contain `:userId` in their path
- D) Only if the developer explicitly calls the validator inside each route

30. What is the main benefit of using `param` middleware over repeating validation inside every route?

- A) It makes the application run faster by caching responses

- B) It follows the DRY principle — one centralized validation that all matching routes benefit from
- C) It automatically sends 400 errors without needing any code
- D) It prevents the need for `express.json()` middleware

Section 5: Request Body & JSON Responses

31. Which middleware MUST be added before routes to allow Express to parse JSON request bodies?

- A) `app.use(express.urlencoded({ extended: true }));`
- B) `app.use(cors());`
- C) `app.use(express.json());`
- D) `app.use(express.text());`

32. A POST request is sent to `/users` with the body `{ "name": "Ali", "age": 22 }`. Which code correctly reads the name?

- A) `const name = req.params.name;`
- B) `const name = req.query.name;`
- C) `const name = req.body.name;`
- D) `const name = req.headers.name;`

33. What is the difference between `express.json()` and `express.urlencoded()`?

- A) `express.json()` parses form data; `express.urlencoded()` parses JSON
- B) `express.json()` parses JSON bodies; `express.urlencoded()` parses HTML form submissions (`key=value&...`)
- C) They are identical — either works for both JSON and form data
- D) `express.urlencoded()` is deprecated and should not be used

34. What HTTP status code should be returned when a new resource is successfully created?

- A) 200
- B) 204
- C) 201
- D) 302

35. Which of the following correctly sends a 400 Bad Request JSON error response?

- A) `res.json(400, { ok: false, error: 'Invalid input' });`
- B) `res.status(400).json({ ok: false, error: 'Invalid input' });`
- C) `res.send({ status: 400, error: 'Invalid input' });`
- D) `res.error(400).json({ ok: false });`

36. Why is request body data preferred over URL query strings for sensitive information like passwords?

- A) Request bodies are encrypted by default in Express
- B) URLs can be logged in server logs, browser history, and proxies — body data is not exposed in the URL
- C) Express cannot read passwords from query strings
- D) Query strings have a maximum length of 10 characters

Section 6: Putting It All Together

37. A route is defined as `app.get('/api/search', handler)`. A request arrives at `/api/search?term=express&limit=5`. Inside the handler, what is `req.params.term`?

- A) `'express'`
- B) `undefined` — `term` is a query parameter, not a route parameter
- C) `5`
- D) An error is thrown because mixed params are not allowed

38. Which of the following complete Express setups is correct and will work properly?

- A) `app.get('/')`; `app.use(cors())`; `app.use(express.json())`; `app.listen(3000)`;
- B) `app.use(cors())`; `app.use(express.json())`; `app.get('/', handler)`; `app.listen(3000)`;
- C) `app.listen(3000)`; `app.use(cors())`; `app.get('/', handler)`;
- D) `app.use(express.json())`; `app.get('/', handler)`; (missing `listen`)

39. A developer writes this route: `app.delete('/users/:userId', (req, res) => { res.json({ deleted: req.params.userId }); });` A request is made to `DELETE /users/abc`. What is the response?

- A) A 404 error because `userId` must be a number
- B) `{ deleted: 'abc' }` — route params are strings by default
- C) An error because `DELETE` routes don't support params
- D) `{ deleted: undefined }`

40. A developer wants to validate `:productId` across multiple routes. Which approach is best?

- A) Copy-paste the validation code into each individual route handler
- B) Use `app.param('productId', validatorFn)` to centralize validation and avoid repetition
- C) Use a global `app.use()` middleware that checks every request URL with regex
- D) Create a separate Express app just for product routes

Answer Key

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8
B	A	B	C	C	B	C	B

Q9	Q10	Q11	Q12	Q13	Q14	Q15	Q16
C	C	D	B	B	C	B	B
Q17	Q18	Q19	Q20	Q21	Q22	Q23	Q24
B	B	B	C	C	D	B	A
Q25	Q26	Q27	Q28	Q29	Q30	Q31	Q32
B	B	B	B	C	B	C	C
Q33	Q34	Q35	Q36	Q37	Q38	Q39	Q40
B	C	B	B	B	B	B	B